Healthcare Risk Prediction — Machine Learning vs Neural Networks

1. Dataset Overview

```
df = pd.read_csv("/content/healthcare dataset.csv")
df.head()
```

	patient_id	age	bmi	exercise_hours_per_week	smoker	blood_pressure	diabetes
0	1	31	18.3	1.5	0	119.1	0
1	2	44	26.3	3.1	0	120.5	0
2	3	43	24.2	4.8	0	130.3	0
3	4	32	25.0	6.0	0	119.5	0
4	5	28	22.1	4.2	1	127.4	0

2. Dataset Structure

```
# a. shape
df.shape
```

```
(6000, 7)
```

```
df.dtypes
```

	0
patient_id	int64
age	int64
bmi	float64
exercise_hours_per_week	float64
smoker	int64
blood_pressure	float64
diabetes	int64

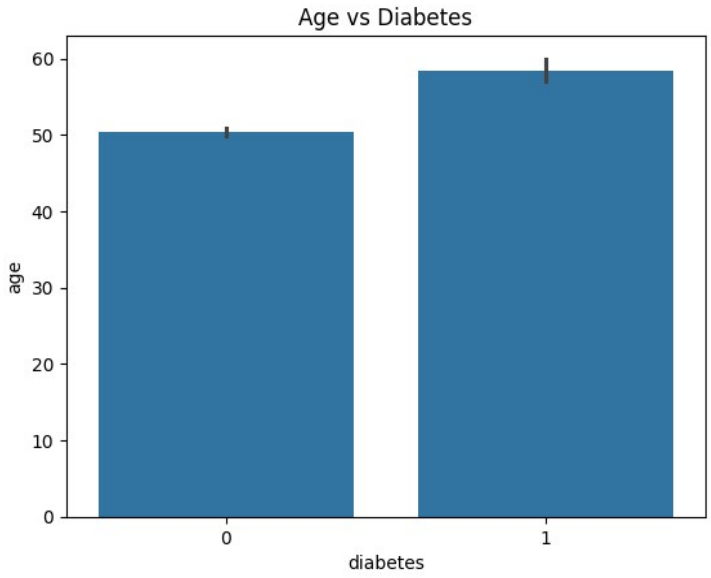
dtype: object

3. Statistical Summary

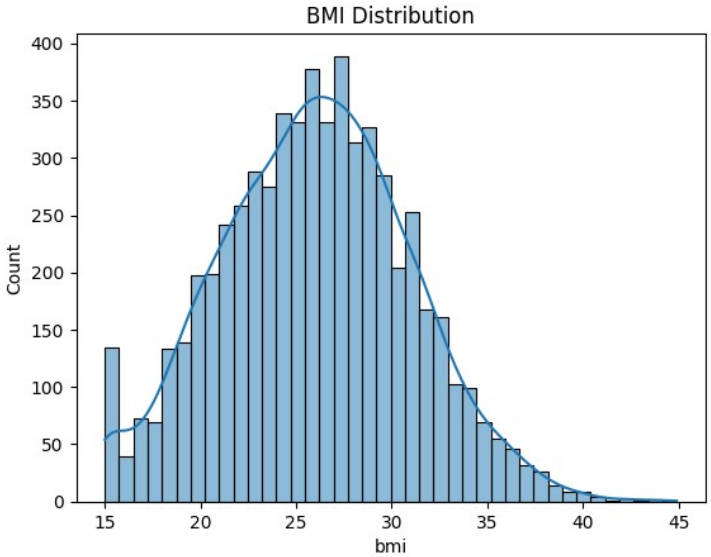
```
df.describe()
```

	patient_id	age	bmi	exercise_hours_per_week	smoker	blood_pressure	diabetes
count	6000.000000	6000.000000	6000.000000	6000.000000	6000.000000	6000.000000	6000.000000
mean	3000.500000	51.107333	25.995267	3.012800	0.307000	134.239733	0.090333
std	1732.195139	19.522722	4.960541	1.491802	0.461288	11.234260	0.286683
min	1.000000	18.000000	15.000000	0.000000	0.000000	94.000000	0.000000
25%	1500.750000	34.000000	22.500000	2.000000	0.000000	126.000000	0.000000
50%	3000.500000	51.000000	26.100000	3.000000	0.000000	134.200000	0.000000
75%	4500.250000	68.000000	29.400000	4.000000	1.000000	142.325000	0.000000
max	6000.000000	84.000000	44.900000	8.500000	1.000000	169.700000	1.000000

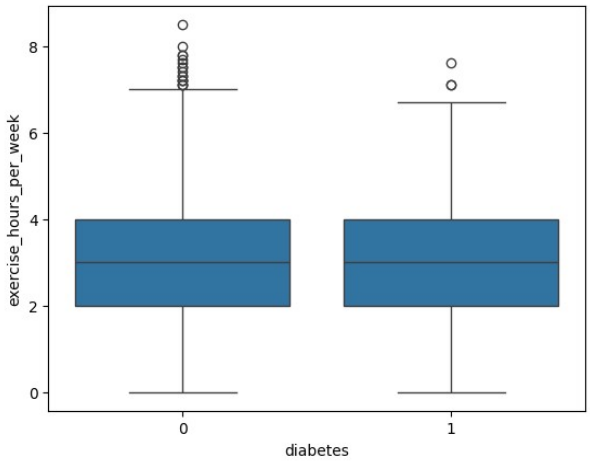
4. Age vs Diabetes Visualization



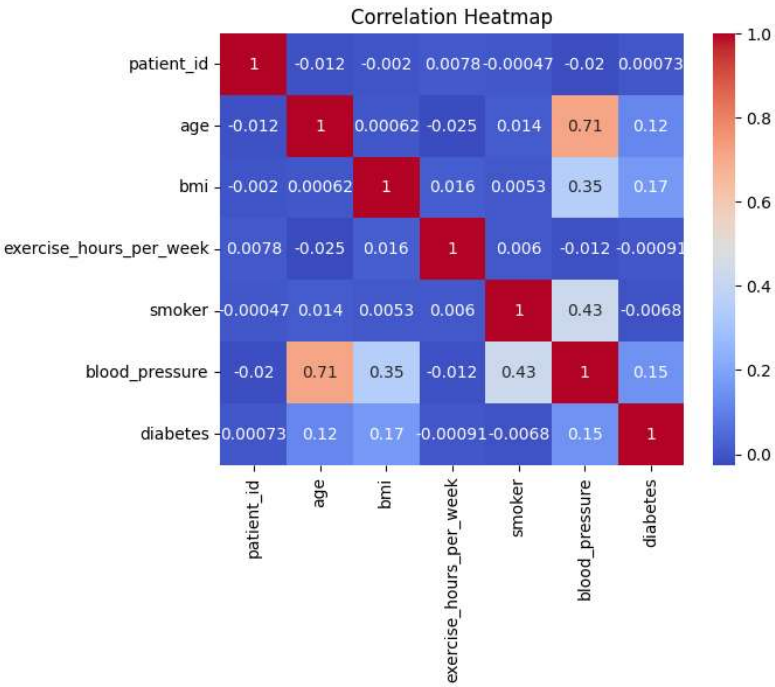
5. BMI Distribution



6. Exercise vs Diabetes



7. Correlation Heatmap



8. Logistic Regression Model Results

```
Accuracy: 0.6533333333333333
Confusion Matrix:
 [[716 376]
 [ 40  68]]
Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.66      0.77      1092
           1       0.15      0.63      0.25       108

    accuracy                           0.65      1200
   macro avg       0.55      0.64      0.51      1200
weighted avg       0.88      0.65      0.73      1200
```

9. Feature Importance

```python
# Feature Importance

coefficients = pd.DataFrame(model.coef_[0], X.columns, columns=['Coefficient'])
print(coefficients)
```

```
                         Coefficient
age                         0.000556
bmi                         0.437111
exercise_hours_per_week     0.007643
smoker                     -0.593564
blood_pressure              0.660961
```

10. Neural Network Architecture

```python
# Display model architecture
nn_model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
hidden_layer_1 (Dense)	(None, 32)	192
hidden_layer_2 (Dense)	(None, 16)	528
output_layer (Dense)	(None, 1)	17

Total params: 737 (2.88 KB)
Trainable params: 737 (2.88 KB)

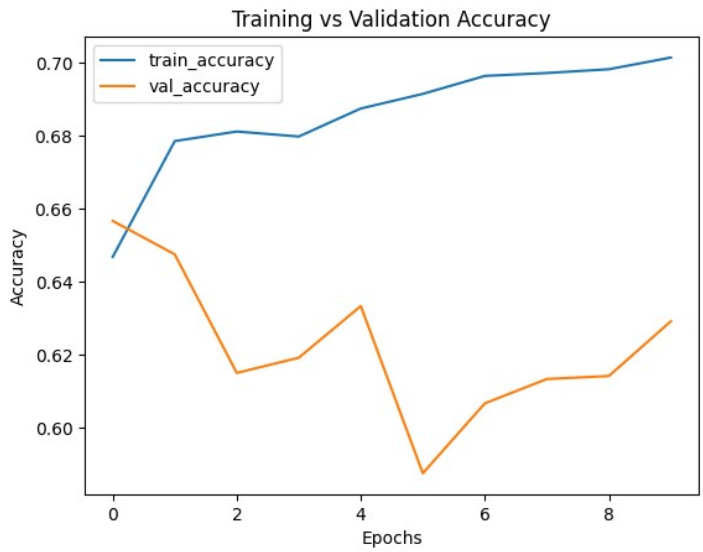
11. Neural Network Evaluation

```
# Evaluate Model
loss, accuracy, mse = nn_model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy)
print("Test MSE:", mse)
```

```
38/38 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - Accuracy: 0.6292 - MSE: 0.2157 - loss: 0.6110
Test Loss: 0.610990047454834
Test Accuracy: 0.6291666626930237
Test MSE: 0.21568697690963745
```

15. Training Performance Graph



The training and validation accuracy curves show that the model learns patterns from the training data over time, as the training accuracy gradually increases. However, the validation accuracy fluctuates and remains lower than the training accuracy, indicating slight overfitting. This suggests that while the model fits the training data reasonably well, its ability to generalize to unseen data is somewhat limited.

16. Model Comparison

```
Logistic Regression Accuracy: 0.6533333333333333
Neural Network Accuracy: 0.6291666626930237
                  Model  Accuracy
0  Logistic Regression  0.653333
1        Neural Network  0.629167
```